Description Submission view

Available from: Cuma, 10 Nisan 2026, 10:50 AM
Due date: Cuma, 10 Nisan 2026, 12:05 PM
Requested files: Main.java (Download)
Type of work: Individual work
Reduction by automatic evaluation: 30 **Free evaluations**: 4**Reduction by automatic evaluation**: 30 **Free evaluations**: 4

Laboratory Manual: Optical Facility Simulation

Welcome to your Object-Oriented Programming laboratory assignment. In this session, you will design the architecture for an optical facility that manipulates laser beams. This exercise is designed to reinforce your understanding of core OOP principles, including Encapsulation, Inheritance, Abstract Classes, Interfaces, Polymorphism, and 2D Array manipulation.

Please read the instructions carefully. Your system will be evaluated using an automated test suite based on strict structural and mathematical requirements. The Main execution file will be provided by the testing system.

Objectives

- Build a robust class hierarchy representing different optical elements.
- Implement interfaces to categorize object capabilities based on energy absorption.
- Manage object states and interactions as they pass through a simulated optical grid (2D array).

Grading Rubric (Total: 100 Points)

Your submission will be evaluated based on the following automated point distribution:

- **Category 1: Basic Methods (15 Points)**
  - `LaserBeam` core functionalities (Constructors, Getters, Setters): 10 tests x 1.0 pt = 10 Points.
  - `OpticalElement` and `EnergyAbsorber` basic features: 5 tests x 1.0 pt = 5 Points.
- **Category 2: Architecture and Type Verification (3 Points)**
  - Inheritance and Interface implementations: 6 tests x 0.5 pt = 3 Points.
- **Category 3: Class Logic and Scenario (42 Points)**
  - `Mirror` Algorithm: 4 tests x 2.0 pts = 8 Points.
  - `FrostedGlass` Algorithm: 6 tests x 2.5 pts = 15 Points.
  - `Prism` Algorithm: 5 tests x 3.0 pts = 15 Points.
  - General Laboratory Scenario validation: 1 test x 4.0 pts = 4 Points.
- **Category 4: System Management (40 Points)**
  - `OpticalFacility` execution, matrix traversal, output format, and mathematical integration: 10 tests x 4.0 pts = 40 Points.

System Architecture Requirements

You are required to create the following components. Ensure that all class names, method names, and parameter types match these descriptions exactly. Do not use import statements unless specifically instructed.

1. The Data Carrier: LaserBeam

This class represents the physical laser beam traveling through your optical grid.

?

- **Attributes:** Define two private fields of type `double`: `angle` and `energy`.
- **Constructor:** Create a constructor that accepts two `double` parameters (in the order of angle, then energy) to initialize the object's state.
- **Behaviors:** Implement standard public getter and setter methods for both attributes: `getAngle`, `setAngle`, `getEnergy`, and `setEnergy`.

2. The Capability Interface: EnergyAbsorber

Some optical elements physically absorb energy from the laser, while others only change its direction. We need a way to group the units that trap energy.

- **Structure:** Create an interface named `EnergyAbsorber`.
- **Method:** Declare a single method named `getAbsorbedEnergy` that takes no parameters and returns a `double`.

3. The Blueprint: OpticalElement

This is the foundational component for every element in the facility.

- **Structure:** Create an abstract class named `OpticalElement`.
- **Attributes:** Define a private `String` field named `name`.
- **Constructor:** Accept a `String` parameter to initialize the `name` attribute.
- **Behaviors:**
  - Provide a public getter method named `getName` returning a `String`.
  - Declare a public abstract method named `processBeam`. This method must return `void` and accept a single parameter of type `LaserBeam` (name the parameter `beam`).

Concrete Optical Units

You will now design the specific physical elements by extending your blueprint.

A. Mirror

- **Inheritance:** Extends `OpticalElement`.
- **Constructor:** Accepts a `String` name and passes it to the parent constructor.
- **Logic (processBeam):** This unit does not absorb energy. It simply reflects the beam by changing its angle. The new angle must be calculated as exactly **180.0** minus the beam's current angle.

B. FrostedGlass

- **Inheritance:** Extends `OpticalElement` and implements `EnergyAbsorber`.
- **Attributes:** Needs a private `double` field named `retainedEnergy`, initialized to 0.0.
- **Constructor:** Accepts a `String` name and passes it to the parent constructor.
- **Logic (processBeam):** This glass diffuses the laser, removing **30% (0.30)** of the beam's current energy. You must subtract this calculated amount from the laser object and add it to the unit's internal `retainedEnergy` tracker. It does not affect the angle.
- **Interface:** Implement `getAbsorbedEnergy` to return the internal tracking variable.

C. Prism

- **Inheritance:** Extends `OpticalElement` and implements `EnergyAbsorber`.
- **Attributes:** Needs a private `double` field named `retainedEnergy`, initialized to 0.0.
- **Constructor:** Accepts a `String` name and passes it to the parent constructor.
- **Logic (processBeam):** This element absorbs **10% (0.10)** of the beam's current energy, transferring it to its `retainedEnergy` tracker. Additionally, it bends the light, meaning it divides the beam's current angle exactly by **1.5**.
- **Interface:** Implement `getAbsorbedEnergy` to return the internal tracking variable.

The Facility Manager: OpticalFacility

This central class manages the physical grid of optical elements.

- **Attributes:**
  - A private 2-dimensional array of `OpticalElement` named `grid`.
  - A private 1-dimensional array of `LaserBeam` named `lasers`.
- **Constructor:** Accepts both arrays (`OpticalElement[][] grid`, `LaserBeam[] lasers`) and assigns them to the class attributes.
- **Method: fireBeams**
  - Returns `void` and takes no parameters.
  - Iterate over the 2D `grid` array row by row.
  - The `LaserBeam` object located at `lasers[i]` must sequentially visit every `OpticalElement` in the corresponding `grid[i]`row.
  - After a laser completes its journey through an entire row, print its final state to the console exactly as follows (where `i` is the row index): `Line [i] Output Angle: [final_angle_value]` `Line [i] Output Energy: [final_energy_value]` (followed by a newline character `\n`).
- **Method: calculateTotalAbsorbedEnergy**
  - Returns a `double` and takes no parameters.
  - Iterate through all units in the entire 2D grid. Identify which units are capable of absorbing energy (use the appropriate OOP keyword). Downcast them appropriately, sum up all their retained energy, and return the final total.

Edge Cases and Boundary Conditions

To achieve a perfect score, your system must handle the following edge cases robustly, as tested by the automated system:

- **Extreme and Zero Inputs:** Your `LaserBeam` class must safely handle extreme values (e.g., 99999.9) and exact zero values (0.0) for both angle and energy without causing calculation errors. Ensure that altering the energy does not unintentionally alter the angle, and vice versa.
- **Zero-State Processing:** If a beam with 0.0 energy enters a `FrostedGlass` or `Prism`, the retained energy should correctly evaluate to 0.0. If a beam with an angle of 180.0 hits a `Mirror`, the new angle must cleanly become 0.0.
- **Cumulative Absorption:** Energy-absorbing units must accurately accumulate retained energy over time. If multiple beams (or the same beam multiple times) pass through a single `FrostedGlass` or `Prism`, its `retainedEnergy`attribute must hold the sum of all absorbed energy.
- **Empty Grid Handling:** If the `OpticalFacility` is initialized with empty arrays for the grid or the lasers, calling `fireBeams` should not crash the program (do not throw out-of-bounds or null pointer exceptions).
- **Pre-Execution State:** If `calculateTotalAbsorbedEnergy` is called before `fireBeams` has been executed, it must safely return 0.0.
- **Row Independence:** In the `OpticalFacility`, a laser beam traveling through one row must not affect the energy or angle of a laser beam traveling through a different row.

Example Scenario for Self-Verification

Before submitting, trace this mathematical scenario on paper to ensure your logic is sound:

1. A `LaserBeam` starts with **100.0** energy and **30.0** angle.
2. It enters a `FrostedGlass` element.
   - It loses 30% of its energy (30.0). The glass stores this 30.0.
   - Laser status: **70.0** energy, **30.0** angle.
3. It immediately enters a `Prism` unit.
   - It loses 10% of its current 70.0 energy (7.0). The unit stores this 7.0.
   - The angle is divided by 1.5 (30.0 / 1.5 = 20.0).
   - Final laser status: **63.0** energy, **20.0** angle.

4. If you call `calculateTotalAbsorbedEnergy()` on this specific system, it should return **37.0** (30.0 + 7.0). Ensure that your system's initial total energy (100.0) is always equal to the final output energy (63.0) plus the total absorbed energy (37.0).

```
--- SIMULATION STARTING ---

Line 0 Output Angle: 160.0
Line 0 Output Energy: 44.1

Line 1 Output Angle: 20.0
Line 1 Output Energy: 81.0

Line 2 Output Angle: 120.0
Line 2 Output Energy: 34.3

--- STATISTICS ---
Total Energy Absorbed by the Entire Facility: 140.6
System Validation (Total Entering Energy 300.0 = Outputs + Absorbed): 300.0
```

Requested files

Main.java

```java
public class Main {
    public static void main(String[] args) {

        LaserBeam[] lasers = {
            new LaserBeam(30.0, 100.0),
            new LaserBeam(45.0, 100.0),
            new LaserBeam(60.0, 100.0)
        };

        OpticalElement[][] facilityGrid = {
            { new FrostedGlass("Glass1"), new Prism("Prism1"), new Mirror("Mirror1"), new FrostedGlass("Glass2") },
            { new Prism("Prism2"), new Prism("Prism3"), new Mirror("Mirror2"), new Mirror("Mirror3") },
            { new FrostedGlass("Glass3"), new FrostedGlass("Glass4"), new FrostedGlass("Glass5"), new Mirror("Mirror4") }
        };

        OpticalFacility facility = new OpticalFacility(facilityGrid, lasers);

        System.out.println("--- SIMULATION STARTING ---\n");
        facility.fireBeams();

        System.out.println("--- STATISTICS ---");
        double totalAbsorbed = facility.calculateTotalAbsorbedEnergy();
        System.out.println("Total Energy Absorbed by the Entire Facility: " + totalAbsorbed);

        double validation = lasers[0].getEnergy() + lasers[1].getEnergy() + lasers[2].getEnergy() + totalAbsorbed;
        System.out.println("System Validation (Total Entering Energy 300.0 = Outputs + Absorbed): " + validation);
    }
}
```

VPL